Studio Nx

JS 23 — initPhotoUpload()

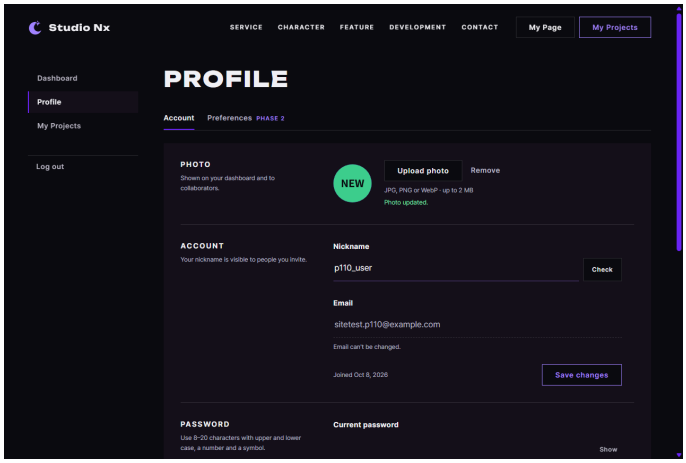
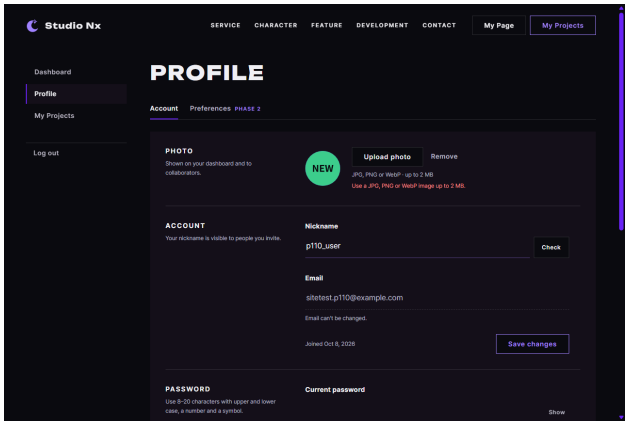
2026-10-12

JS 23 — `initPhotoUpload()`

항목	내용
파일	`studionx/frontend/static/js/pages/userinfo.js`, `userInfo.html`(한 줄), `static/css/pages/userinfo.css`(세 줄)
작업	P1-10 회원 정보 화면 JS: 프로필 사진 올리기와 지우기 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	22 `loadMe()` · 24 `initProfileForm()`

1. 하는 일

Upload photo로 고른 사진을 서버에 올리고, **Remove**로 지웁니다. 결과는 사진 아래에 한 줄로 알립니다.

올린 사진	거절 (GIF)
	

한 일	화면
PNG, JPG, WebP (2MB 이하)를 고름	바로 "Uploading..."이 나오고 버튼이 잠긴 뒤, 동그라미가 새 사진으로 바뀌고 초록 "Photo updated."가 나옵니다
GIF나 2MB가 넘는 파일을 고름	**보내지 않고** 빨강 "Use a JPG, PNG or WebP image up to 2 MB."를 보여 줍니다. 동그라미는 그대로입니다
확장자는 `.jpg`인데 내용은 사진이 아님	서버가 415로 응답하고 빨강 "Use a JPG, PNG or WebP image."가 나옵니다 (P1-08 38의 내용 검사)
Remove	닉네임 첫 글자로 돌아가고, 초록 "Photo removed."가 나옵니다
같은 파일을 다시 고름	다시 올라갑니다

2. 언제 쓰나

userInfo.html이 열릴 때 한 번 부릅니다 (`initPhotoUpload();`). 22번 `loadMe`가 처음 동그라미를 그리고, 이 함수는 사진을 바꾼 뒤 22번의 `renderAvatar`로 다시 그립니다.

쓰는 API	서버
`POST /api/v1/users/me/profile-image` (FormData, 칸 이름 `file`)	백엔드 42 → 38

쓰는 API	서버
`DELETE /api/v1/users/me/profile-image`	42 → 39

HTML과 CSS에 더한 것 (AI 화면 작업)

파일	내용
`userInfo.html`	사진 버튼, 안내, 문구를 감싼 `<div>`에 `id="photo-field"`를 붙였습니다. `setFieldState`가 상태 클래스를 붙일 곳입니다
`userinfo.css`	`#photo-field.is-ok`, `.is-error`일 때 `#photo-msg`를 초록, 빨강으로 바꾸고 위 여백을 줍니다. 이 문구는 폼 칸(`.field`) 밖에 있어서 폼 CSS의 색이 닿지 않았기 때문입니다

3. 코드

import (파일 맨 위: 22번과 같고, 추가한 것은 없습니다). 이 함수는 **`api`**, **`setFieldState`**, 22번의 **`renderAvatar`**, 템플릿의 **`PHOTO_TYPES`와 `PHOTO_MAX_BYTES`**를 씁니다.

함수

```
export function initPhotoUpload() {
  const input = document.getElementById('photo-input');
  const remove = document.getElementById('photo-remove');
  const field = document.getElementById('photo-field');
  if (!input) return;

  const setBusy = (busy) => {
    input.disabled = busy;
    if (remove) remove.disabled = busy;
  };

  input.addEventListener('change', async () => {
    const file = input.files && input.files[0];
    if (!file) return; // 고르다가 취소
    if (PHOTO_TYPES.indexOf(file.type) < 0 || file.size > PHOTO_MAX_BYTES) {
      setFieldState(field, 'error', 'Use a JPG, PNG or WebP image up to 2 MB.');
      input.value = '';
      return;
    }
    const form = new FormData();
    form.append('file', file);
    setBusy(true);
    setFieldState(field, null, 'Uploading…');
    try {
      renderAvatar(await api.post('/users/me/profile-image', form));
      setFieldState(field, 'ok', 'Photo updated.');
    } catch (err) {
      setFieldState(field, 'error', err.message);
    } finally {
      setBusy(false);
      input.value = '';
    }
  });

  if (remove) {
```

```
    remove.addEventListener('click', async () => {
      setBusy(true);
      try {
        renderAvatar(await api.delete('/users/me/profile-image'));
        setFieldState(field, 'ok', 'Photo removed.');
      } catch (err) {
        setFieldState(field, 'error', err.message);
      } finally {
        setBusy(false);
      }
    });
  }
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const file = input.files?.[0];
if (!PHOTO_TYPES.includes(file.type) || file.size > PHOTO_MAX_BYTES) { … }
…
remove?.addEventListener('click', async () => { … });
```

최신 문법	실제 코드	뜻
`input.files?.[0]`	`input.files && input.files[0]`	목록이 있을 때만 첫 번째를 꺼냅니다
`arr.includes(x)`	`arr.indexOf(x) >= 0` (여기서는 "없으면"이므로 `< 0`)	목록에 있는지 확인합니다
`remove?.addEventListener(…)`	`if (remove) { remove.addEventListener(…) }`	버튼이 있을 때만 연결합니다

문법 풀이

식	뜻
`'change'` (파일 칸)	파일을 고르고 확인했을 때 일어납니다. 같은 파일을 다시 고르면 값이 같아서 **일어나지 않으므로**, 끝날 때 `input.value = ''`로 비웁니다
`input.files[0]`, `file.type`, `file.size`	고른 파일, 브라우저가 알려 준 종류(`image/png`), 바이트 수입니다
`new FormData()`, `form.append('file', file)`	파일 올리기용 본문입니다. `api.js`는 FormData이면 종류 헤더를 비워 두므로, 브라우저가 경계(boundary)를 붙입니다
`renderAvatar(await api.post(…))`	서버 응답(내 정보)을 받자마자 그 사진 주소로 다시 그립니다
`try … catch … finally`	`finally`는 성공해도 실패해도 실행됩니다. 잠금 풀기를 한 곳에서 할 수 있습니다
`<label for="photo-input">` (HTML)	진짜 파일 칸은 숨겨 두고, 버튼 모양의 이름표를 누르면 파일 고르기 창이 열립니다. 칸이 `disabled`이면 이름표를 눌러도 열리지 않습니다

4. 흐름

파일을 골랐을 때: 취소했으면 끝냅니다. 종류나 크기가 틀리면 빨간 문구를 보여 주고 끝냅니다. 맞으면 "Uploading…"을 보여 주고 버튼을 잠근 뒤 올립니다. 성공하면 새 사진과 초록 문구를, 실패하면 서버 문구를 빨

강으로 보여 줍니다. 마지막에 잠금을 풀고 칸을 비웁니다. **Remove를 눌렀을 때:** 버튼을 잠그고 사진을 지웁니다. 성공하면 첫 글자와 초록 문구를, 실패하면 빨간 문구를 보여 주고 잠금을 풉니다.

5. 왜 이렇게

선택	이유
화면에서 먼저 종류와 크기를 검사	10MB 사진을 다 올린 뒤에야 거절당하지 않게 합니다. 서버도 같은 기준으로 다시 검사합니다 (브라우저가 알려 준 종류는 믿을 수 없기 때문입니다, 38)
화면 문구는 하나 ("… up to 2 MB.")	종류가 틀리든 크기가 틀리든 고치는 방법이 같으므로, 조건을 한 번에 알려 줍니다
서버가 거절하면 서버 문구를 그대로 보여 줌	화면 검사를 통과했는데 내용이 사진이 아닌 경우는 서버만 알 수 있습니다
"Uploading…"과 버튼 잠금	느린 망에서 두 번 올리거나, 올리는 중에 지우는 일을 막습니다
성공 문구 (템플릿에는 없었음)	동그라미만 바뀌면 화면 읽기 프로그램 사용자는 알 수 없습니다. `#photo-msg`는 `aria-live` 칸입니다
사진이 없을 때도 Remove를 둠	서버가 200을 줍니다(39). 숨기면 사진 유무에 따라 화면이 흔들립니다

6. 시험 (실제 userInfo.html, 실제 서버, 헤드리스 Chrome, 9개)

사진은 페이지 안에서 canvas로 만들고, `DataTransfer`로 파일 칸에 넣은 뒤 `change`를 보냈습니다 (사람이 고른 것과 같은 이벤트입니다).

시험	확인한 것
PNG 올리기	보내는 중에 "Uploading…"이 나오고 두 버튼이 잠깁니다. 이어서 동그라미가 `<img src="/api/v1/profile-images/{32자}.png">`로 바뀌고 "Photo updated."가 초록(`rgb(127, 224, 168)`)으로 나오며, 잠금이 풀리고 칸이 비워집니다
GIF, 2MB+1바이트	빨강(`rgb(255, 138, 138)`) 문구가 나오고, **요청은 0번**이며, 동그라미는 앞의 사진 그대로입니다
`.jpg`인데 내용이 HTML	요청을 1번 보내고, 서버의 415 문구 "Use a JPG, PNG or WebP image."가 나옵니다
Remove	첫 글자 `P`와 "Photo removed."가 나옵니다
같은 파일을 다시 고름	다시 올라갑니다

9개 모두 통과했습니다. 22번 시험(8개)도 다시 돌려서 통과했습니다.